

Broadcasting

In questo esempio verrà mostrato il modo in cui la soluzione di un problema sui sistemi distribuiti viene formalizzata

- Descrizione in formale
- Assunzioni e restrizioni del sistema
- Definizione dell'insieme degli stati possibili
- Definire l'insieme degli stati iniziali, transitori e finali del sistema
- Definizione di $B(x) \forall x \in E \rightsquigarrow$ solitamente coincide con la descrizione del protocollo
- Analisi della temporal/messaggio complexity

Descrizione informale

Un nodo sorgente invia un messaggio a tutti gli altri nodi ed il protocollo termina quando tutti gli altri nodi hanno ricevuto il messaggio

Assunzioni e restrizioni

Assumiamo ora che il sistema abbia le seguenti proprietà:

- a) Total reliability
- b) Connectivity
- c) Bidirectional links $\rightsquigarrow$ per semplicità
- d) Unique initiator, c'è un solo nodo x che riceve il messaggio iniziale

Insieme degli stati possibili

I possibili stati sono

- Init:

Lo stato in cui si trova l'unico nodo che riceve il messaggio da trasmettere

- Idle:

Stato in cui si trovano i nodi in attesa di ricevere il messaggio

- Done:

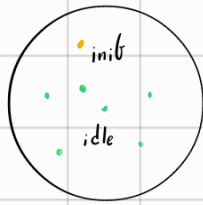
Stato in cui si trovano i nodi che hanno ricevuto il messaggio

Per cui avremo un insieme di stati possibili $S = \{\text{Init}, \text{Idle}, \text{Done}\}$

Stati iniziali, transitori e finali

• Stati iniziali:

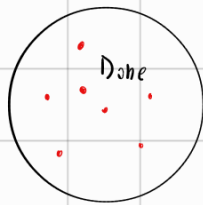
$\exists! x \in C.c. \text{ status}(x) = \tilde{\text{init}}$ and $\forall y \in E, x \neq y \in C.c. \text{ status}(y) = \tilde{\text{idle}}$



$\Rightarrow$ Abbiamo n configurazioni iniziali possibili

• Stati finali

$\forall x \in C.c. \text{ status}(x) = \text{Done}$



$\Rightarrow$ Abbiamo 1 configurazione finale possibile

Definizione del Behaviour $\leadsto$ stiamo parlando ora del protocollo vero e proprio

Flooding

1) $\tilde{\text{init}} \times i$:

$\text{send}(\tilde{I}) \text{ to } N(x) \leadsto \tilde{I} = \text{messaggio}, i \text{ indica un impulso spontaneo}$
 become Done

2) $\tilde{\text{idle}} \times \text{receiving}(\tilde{I})$:

$\text{memorizza}(\tilde{I})$

$\text{send}(\tilde{I}) \text{ to } N_x \setminus \{\text{sender}\}$

become Done

3) $\tilde{\text{init}} \times \text{receiving}(\tilde{I})$:

N_{ull}

4) $\tilde{\text{idle}} \times i$:

N_{ull}

5) $\text{Done} \times \text{receiving}(\tilde{I})$:

N_{ull}

6) $\text{Done} \times i$:

N_{ull}

Protocollo

Flooding:

- Status Values:

$$S = \{\tilde{Init}, \tilde{Idle}, Done\}$$

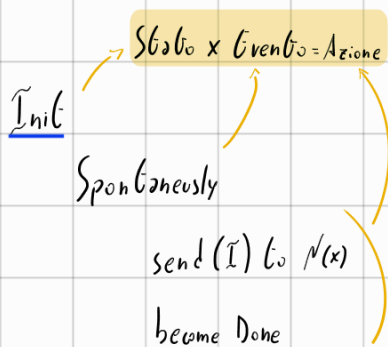
$$S_{init} = \{\tilde{Init}, \tilde{Idle}\}$$

$$S_{term} = \{Done\}$$

- Restrictions:

Bidirectional links, Total reliability

Connectivity, Unique initiator



$\tilde{Idle}$

receiving $(\tilde{I})$

Process $(\tilde{I})$

send $(\tilde{I})$ to $N(x)$ - sender

become Done

Dimostrazione di correttezza

Utilizzando la total reliability dimostriamo che l'algoritmo e' corretto tramite un'induzione sul diametro di G

↳ Stiamo dimostrando che tempo di completamento sia finito

Lemma

$\forall h$ diametro di G $\exists T \in \mathbb{R}^+$:

tutti i nodi a distanza h dal nodo sorgente s riceveranno $\tilde{I}$ in tempo $\leq T$

Dim induttiva su h

- Passo base $h=1$

Strutturando la total reliability $\exists T_1$ c.c.

il messaggio $\tilde{I}$ inviato da s arriva a tutti i nodi $x \in N(s)$ in tempo $\leq T_1$

- Passo induttivo $h+1$

$\forall x \in E$ c.c. $d(s, x) = h+1 \exists z \in E / \{x\}$ c.c. $d(s, z) = h$

Ma per il passo induttivo z ha ricevuto il messaggio $\tilde{I}$ in tempo $\tilde{T}_h$

Quindi z secondo $B(z)$ invia il messaggio a $N(z) / \{\text{sender}\}$

Allora abbiamo 2 possibilità:

Ⓐ x è il sender e quindi già informato ✓

Ⓑ x riceve $\tilde{I}$ al tempo $\tilde{T}_{h+1}$ inviato da z all'istante $\tilde{T}_h$ ✓

Strutturando ora l'assunzione di connettività sappiamo che il diametro $h \leq n-1$

Quindi per il lemma sappiamo che l'algoritmo termina sicuramente in tempo $\tilde{T}_n$ finito

↳ Visto che una volta ricevuto il messaggio il nodo va nello stato Done, essendo uno stato di quietanza, abbiamo dimostrato anche il tempo di terminazione

Analisi Message complexity

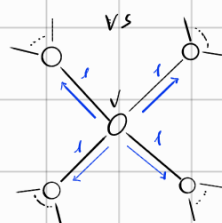
L'idea dietro quest'analisi è:

Fissato un'arco in G , quanti messaggi ci passano attraverso?

Analizziamo ora i seguenti casi:

- a) Un endpoint dell'arco è la sorgente:

In questo caso sugli archi passano un solo messaggio



- b) Entrambi gli endpoint sono diversi dalla radice:

v riceve $\tilde{I}$ al tempo $\tilde{T}_v$ e manda $\tilde{I}$ a $N(v) / \{\text{sender}\}$

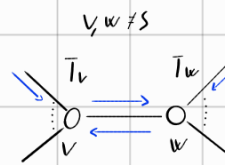
Ma w riceve $\tilde{I}$ da un nodo $\neq v$ in tempo $\tilde{T}_w = \tilde{T}_v + \epsilon$ e quindi manda $\tilde{I}$ a $N(w) / \{\text{sender}\}$ con $\text{sender} \neq w$

Quindi sull'arco (v, w) passano 2 messaggi: $\vec{v, w}$ e $\vec{w, v}$

Quindi nel caso peggiore

Abbiamo 2 $\tilde{I}$ per arco quindi

$$Msg(x) = 2m = \mathcal{O}(m)$$



Un altro tipo di analisi passa per quante copie di $\tilde{I}$ un nodo manda:

$$N(s) + \sum_{x \in V \setminus \{s\}} N(x) - 1 = N(s) + \sum_{x \in V \setminus \{s\}} N(x) - (n-1) = \sum_{x \in V} N(x) - (n-1) = 2m - (n-1) = \mathcal{O}(m)$$

Analisi temporal complexity

Grazie alla total reliability sappiamo che

$\exists \Delta_t$ entro il quale ogni messaggio inviato arriva in tempo $\leq \Delta_t$

Possiamo quindi normalizzare in base a Δ_t così da rendere questa quantità = 1

Calcoliamo ora quanto è lungo il cammino minimo massimo dalla sorgente s ad $x \in V$

$\max \{d(s, x)\}$ = distanza massima di $x \in V$ da $s \leq \text{diametro}(G) = n-1$

Per cui

$$\tilde{\text{time}}(x) = \mathcal{O}(n)$$

Lowerbound sulla message complexity

Il Lower bound della message complexity per il problema del podcast è:

$$Msg(x) = \Omega(m)$$

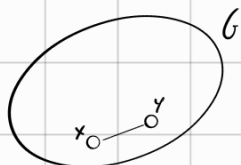
Dim

Sia A un algoritmo che completa il broadcast utilizzando un numero $m \leq m$ di messaggi scambiati

Sia G la topologia di un sistema distribuito su cui far girare A

Siano $x, y \in V$

$\exists e = (x, y)$ su cui A non fa passare alcun messaggio

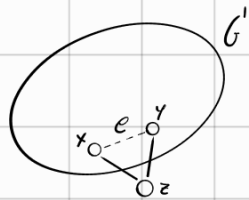


Costruiamo un nuovo grafo G' c.

$$G' = (V \cup \{z\}, E \setminus \{e\} \cup \{(x,z), (y,z)\})$$

Possiamo notare che i 2 nodi x, y non riescano a distinguere i 2 Environment G e G'

Simuliamo ora la stessa esecuzione di A su G'



Visto che x ed y non hanno mai inviato messaggi in G

Allora x ed y non invieranno mai dei messaggi a z

Per cui z non verrà mai indovinato.

Assurdo!!

Abbiamo assunto che A sia completo per il broadcast ma abbiamo presentato un'istanza per cui non sia così